Виды генераторов I

- Физические датчики случайных чисел (используют случайные события, например движения мыши, квантовые явления типа дробового шума, термального эффекта, фотоэлектрического шума и т.д.)
- Алгоритмические датчики псевдослучайных чисел
  Как правило, эти последовательности образуются по некоторым функциональным закономерностям (вычислительным алгоритмам) вида

$$X_{n+1} = f(X_n), \quad n = 0, 1, \ldots$$

или

$$X_{n+k} = f(X_{n+k-1}, X_{n+k-2}, \ldots, X_n), \quad n = 0, 1, \ldots,$$

где числа X_0 или X_0, X_1, ..., X_{k-1} заданы.

© А.В. Зорин, ННГУ, 2021

Виды генераторов II

Definition

Для применений важно иметь последовательность *независимых равномерно распределенных в интервале* $(0,1)$ случайных величин $\boldsymbol{U}_0$, $\boldsymbol{U}_1$, $\ldots$, $\boldsymbol{U}_n$, $\ldots$.
Независимость и равномерное распределение означают следующее: для целого $n \geq 0$ и произвольных подынтервалов Δ_0, Δ_1, $\ldots$ интервала $(0,1)$ имеет место равенство

$$\mathbf{P}(\{\boldsymbol{U}_0 \in \Delta_0, \boldsymbol{U}_1 \in \Delta_1, \ldots, \boldsymbol{U}_n \in \Delta_n\}) = |\Delta_0| \cdot |\Delta_1| \cdots |\Delta_n|.$$

© А.В. Зорин, ННГУ, 2021

Пример: генератор с иррациональным числом I

Пусть α — некоторое число из интервала $(0, 1)$, и образуем последовательность по формуле:

$$X_n = \{n\alpha\}.$$

Здесь $\{\cdot\}$ — дробная часть стоящего в скобках числа.
Г. Вейлем была доказана замечательная теорема о том, что для иррационального α так определенная последовательность чисел равномерно распределена на $(0, 1)$. Под равномерной распределенностью понимается следующее: для любых величин y_1 и y_2, $0 \le y_1 < y_2 \le 1$,

$$\lim_{n\to\infty}\left[\frac{1}{n} \times \mathcal{N}(\{i\colon y_1 \le X_i \le y_2; i = 1, 2, \ldots, n\})\right] = y_2 - y_1,$$

© А.В. Зорин, ННГУ, 2021

Пример: генератор с иррациональным числом II

где $\mathcal{N}(\cdot)$ обозначает мощность конечного множества, указанного в скобках.

[пример в R]

Кроме одномерных распределений следует изучить совместные распределения стоящих рядом членов последовательности. Если соседние члены последовательности «независимы», то точки с координатами (X_{n+1}, X_n), $n = 0, 1, \ldots$ должны быть равномерно рассеяны внутри единичного квадрата.

[пример в R]

Наряду со статистическим обследованием предлагаемого функционального преобразования полезно провести и теоретический анализ. Он выявит, что генератор, основанный на этом принципе, обладает двумя существенными недостатками.

© А.В. Зорин, ННГУ, 2021

Пример: генератор с иррациональным числом III

Во-первых, соседние члены последовательности не являются статистически независимыми. Заметим, что

$$X_{n+1} = \begin{cases} \alpha + X_n & \text{при } X_n \le 1 - \alpha, \\ X_n + \alpha - 1 & \text{при } X_n > 1 - \alpha. \end{cases}$$

Таким образом, величины X_n и X_{n+1} функционально зависимы. На рисунке точки располагаются на прямых

$$y = x + (\sqrt{2} - 1), \qquad y = x + \sqrt{2}.$$

Рассмотрим случайные величины $\boldsymbol{Y}$ и $\boldsymbol{Z}$, удовлетворяющие функциональному соотношению

$$\boldsymbol{Z} = \begin{cases} \alpha + \boldsymbol{Y} & \text{при } \boldsymbol{Y} \le 1 - \alpha, \\ \alpha - 1 + \boldsymbol{Y} & \text{при } \boldsymbol{Y} > 1 - \alpha, \end{cases}$$

© А.В. Зорин, ННГУ, 2021

Пример: генератор с иррациональным числом IV

и предположим, что $\boldsymbol{Y}$ равномерно распределен в интервале $(0,1)$. Тогда $\boldsymbol{Z}$ также имеет равномерное распределение в интервале $(0,1)$. Можно считать, что пара (X_n, X_{n+1}) является выборочной реализацией для $(\boldsymbol{Y}, \boldsymbol{Z})$. Вычислим ковариацию между $\boldsymbol{Y}$ и $\boldsymbol{Z}$:

$$\operatorname{cov}(\boldsymbol{Y}, \boldsymbol{Z}) = \mathsf{M}\left(\boldsymbol{Y} - \frac{1}{2}\right)\left(\boldsymbol{Z} - \frac{1}{2}\right) = \int\limits_0^{1-\alpha} y(y+\alpha)\,dy +$$

$$+ \int\limits_{1-\alpha}^{1} y(y+\alpha-1)\,dy - \frac{1}{4} = \frac{1}{12} - \frac{1}{2}\alpha(1-\alpha).$$

© А.В. Зорин, ННГУ, 2021

Пример: генератор с иррациональным числом V

Ковариация не обращается в нуль, если α не есть $\frac{1}{2} \pm \frac{1}{2\sqrt{3}}$. Второй же недостаток в том, что при вычислении на компьютере вещественные числа представляются неточно и в определенный момент последовательность может выродиться. Действительно, для больших значений n представления с плавающей точкой для $n\alpha$ и $(n+1)\alpha$ не различаются, либо различаются несколькими младшими битами. Тогда члены последовательности будут принимать только значения вида 0, 0,5, 0,25, и т. д.

© А.В. Зорин, ННГУ, 2021

Пример: объем четырехмерной сферы I

Найдем объем V 4-х мерной сферы

$$\{(x_1, x_2, x_3, x_4)\colon x_1^2 + x_2^2 + x_3^2 + x_4^2 \le 1\}.$$

Пусть X_1, X_2, X_3, X_4 — независимые одинаково распределенные величины в интервале $(-1, 1)$. Тогда

$$\mathbf{P}(X_1^2 + X_2^2 + X_3^2 + X_4^2 < 1) = \frac{V}{2^4},$$

откуда

$$V = 16 \times \mathbf{P}(X_1^2 + X_2^2 + X_3^2 + X_4^2 < 1) \approx$$
$$\approx 16 \times \text{частота попадания точки в сферу}$$

Для генерации точки воспользуемся четырмя разными последовательностями (при разных α). Истинный объем четырехмерной сферы $\pi^2/2 \approx 4.934\ldots$

© А.В. Зорин, ННГУ, 2021

Пример: объем четырехмерной сферы II

```
alfa <- c( 1/sqrt(2), (3-sqrt(5))/7,
    pi-3, exp(-1) )
alfa <- c( 1/sqrt(5), (3-sqrt(5))/7,
    1/pi, exp(-1) )

f <- function(x){
  if (sum(x^2)<1)  1 else  0
}

s <- 0
NN <- 100000
for(n in 1:NN){
  s <- s + f( 2*(alfa*n - floor(alfa*n))-1 )
}
16*s/NN
pi^2/2
```

© А.В. Зорин, ННГУ, 2021

Пример: объем четырехмерной сферы III

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор I

В *линейном конгруэнтном генераторе* последовательность псевдослучайных чисел образуется по закону

$$X_{n+1} = aX_n + c \mod m, \tag{2}$$

где модуль m, множитель a, приращение c и начальное значение X_0 — целые числа, удовлетворяющие условиям $0 \leq a < m$, $0 \leq c < m$, $0 \leq X_0 < m$. Задавая различные начальные значения X_0, мы будем получать различные последовательности. Выбор модуля m чаще всего определяется разрядностью целых чисел.
Частным случаем линейного конгруэнтного генератора является *мультипликативный конгруэнтный генератор*, определяемый соотношением (2) при $c = 0$.

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор II

При $a = 1$ генерируемая соотношением (2) последовательность имеет вид

$$X_n = (X_0 + cn) \mod m$$

и, очевидно, плохо подходит на роль «псевдослучайной». Поскольку величина X_n, определяемая по формуле (2), может принимать только конечное число значений, а значение X_n зависит только от значения X_{n-1}, последовательность $\{X_n, n = 0, 1, \ldots\}$ с необходимостью периодическая. Ее период не превосходит m. Длина периода может зависеть как от параметров a, c, n, так и от начального значения X_0.

Выбор параметров следует производить таким образом, чтобы период был наибольший из возможных.

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор III

Theorem (Д.Кнут, TAOCP, v. 2)

Линейная конгруэнтная последовательность

$$X_{n+1} = aX_n + c \mod m,$$

определенная числами m, a, c и X_0, имеет период длиной m тогда и только тогда, когда

1. *числа c и m взаимно простые;*
2. *$b = a - 1$ кратно p для каждого простого p, являющегося делителем m;*
3. *b кратно 4, если m кратно 4.*

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор IV

Definition

Пусть a и m — взаимно простые числа. Наименьшее число λ, для которого $a^\lambda \equiv 1 (\mathrm{mod}\ m)$, называется *порядком a по модулю m* и обозначается $\lambda(m)$.

Theorem (Д.Кнут, TAOCP, v. 2)

Максимальным периодом при $c = 0$ является $\lambda(m)$. Этот период достигается, если

1. *X_0 и m — взаимно простые числа;*
2. *a является первообразным элементом по модулю m.*

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор V

Пусть теперь число d, $1 < d < m$, есть делитель числа m. Тогда величины $Y_n = X_n \mod d$ удовлетворяют соотношению

$$Y_{n+1} = aY_n + c \mod d.$$

В частности, если $m = 2^k$ и $d = 2^l$, $l < k$, то Y_n представляет собой l младших битов числа X_n в k-разрядной двоичной арифметике. Тогда период последовательности $\{Y_n, n = 0, 1, \ldots\}$ не превосходит d, что гораздо меньше максимально возможного периода исходной последовательности $\{X_n, n = 0, 1, \ldots\}$. В этом случае говорят, что младшие биты «менее случайны». Поэтому линейные конгруэнтные генераторы не очень хорошо подходят для получения длинных

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор VI

«непредсказуемых» последовательностей чисел из множества $\{0, 1, \ldots, d-1\}$.

Listing 3: (Реализация мультипликативного генератора на языке R с демонстрацией доп.возможностей языка)

```
local({
.Mod <- 1024;
.lcg_seed <- 1L
.a <-16807
lcg_init <<- function(seed) {
  .lcg_seed <<- seed}
lcg_next.int <<- function() {
  .lcg_seed <<- ((.a*.lcg_seed) %% .Mod)
  .lcg_seed
}
lcg_next.float <<- function() {
```

© А.В. Зорин, ННГУ, 2021

Линейный конгруэнтный генератор VII

```
    lcg_next.int() / .Mod }
})
lcg_next.int()

N <-100000;
x <- rep(0, N)
for (i in 1:N) {x[[i]] <- lcg_next.float()}
hist(x, pr=T, col="red")
ks.test(x, punif)
plot(acf(x))
plot( x[-N], x[-1] )
```

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор I

Пусть имеется векторный компьютер с длиной вектора k. Из соотношения

$$X_{n+1} = aX_n + c \mod m,$$

для произвольного $l = 1, 2, \ldots$ имеем

$$X_{n+l} = a^l X_n + (a^{l-1} + a^{l-2} + \ldots + 1)c \mod m. \qquad (3)$$

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор II

Первый векторный линейный конгруэнтный генератор с единым набором параметров

Для заданных X_0, X_1, ..., X_{k-1} положим

$$\begin{pmatrix} X_n \\ X_{n+1} \\ \vdots \\ X_{n+k-1} \end{pmatrix} = a^k \begin{pmatrix} X_{n-k} \\ X_{n-k+1} \\ \vdots \\ X_{n-1} \end{pmatrix} + \frac{a^k - 1}{a - 1} c \mod m, \quad n = k, 2k, \ldots$$

Второй векторный линейный конгруэнтный генератор с единым набором параметров

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор III

Для заданного X_0 определим

$$\begin{pmatrix} X_n \\ X_{n+1} \\ \vdots \\ X_{n+k-1} \end{pmatrix} = X_{n-1} \begin{pmatrix} a \\ a^2 \\ \vdots \\ a^k \end{pmatrix} + \begin{pmatrix} c \\ (a+1)c \\ \vdots \\ \frac{a^k-1}{a-1}\, c \end{pmatrix} \mod m, \quad n = k, 2k, \ldots$$

Listing 4: Линейный конгруэнтный генератор

```
#define a0 16807

typedef int v4si
   __attribute__ ((vector_size(16)));

union i4vec{
```

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор IV

```
  v4si v;
  int i[4];
};

class lcr{
  union i4vec seed, a;
  int loc;
public:
  lcr(int _seed);
  int next();
};

lcr::lcr(int _seed)
  :loc(0)
{
  const v4si m = {0x7fffffff,0x7fffffff,
```

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор V

```
                    0x7fffffff,0x7fffffff};
  seed.i[0]= (a0*_seed & 0x7fffffff);
  a.i[0] = a0;
  for(int j=1; j<4; j++) {
      a.i[0] = ((a.i[0]*a0) & 0x7fffffff);
      seed.i[j]=(a0*seed.i[j-1] & 0x7fffffff);
    };
  a.i[1]=a.i[2]=a.i[3]=a.i[0];
  seed.v=(a.v*seed.v) & m;
};

inline int lcr::next() {
  const union i4vec m = {0x7fffffff,0x7fffffff,
                         0x7fffffff,0x7fffffff};
  if (++loc == 4) {
    seed.v=(a.v*seed.v) & m.v;
```

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор VI

```
    loc = 0;
  };
  return seed.i[loc];
};
```

Возможность производить векторные операции приводит к следующей модификации линейного конгруэнтного генератора.

Второй векторный линейный конгруэнтный генератор с несколькими наборами параметров

© А.В. Зорин, ННГУ, 2021

Векторный линеный конгруэнтный генератор VII

Пусть $a_1, a_2, \ldots, a_k$ и $c_1, c_2, \ldots, c_k$ — наборы различных множителей и приращений; для заданных $X_0, X_1, \ldots X_{k-1}$ определим

$$\begin{pmatrix} X_n \\ X_{n+1} \\ \vdots \\ X_{n+k-1} \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \end{pmatrix} \odot \begin{pmatrix} X_{n-k} \\ X_{n-k+1} \\ \vdots \\ X_{n-1} \end{pmatrix} + \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{pmatrix} \mod m, \ n = k, 2k, \ldots,$$

где $\odot$ обозначает поэлементное умножение двух векторов.

© А.В. Зорин, ННГУ, 2021

Обобщенный генератор Фибоначчи с запаздыванием I

Обобщенный генератор Фибоначчи с запаздыванием задается соотношением

$$X_n = X_{n-i} \circ X_{n-j} \mod m, \quad n \geq \max\{i, j\}, \tag{4}$$

где $\circ$ означает некоторую бинарную арифметическую или логическую операцию (например, сложение $+$, вычитание $-$, умножение $\cdot$, исключающее «или» $\oplus$).
Параметры i, j называются *запаздываниями*.
При $i = 1$, $j = 2$ и $\circ$ в значении $+$ получается собственно *генератор Фибоначчи*. При $\circ$ в значении $+$ имеем *генератор Фибоначчи с запаздыванием*.

© А.В. Зорин, ННГУ, 2021

Обобщенный генератор Фибоначчи с запаздыванием II

Достоинствами обобщенных генераторов Фибоначчи с запаздыванием является отсутствие умножений и возможность перейти к работе с вещественными числами.

Пусть теперь операция $\circ$ означает сложение или вычитание. Положим $U_n = X_n/m$. Тогда, разделив обе части равенства (4) на m, получим:

$$U_n = \{U_{n-i} \pm U_{n-j}\}, \quad n \geq \max\{i, j\}, \tag{5}$$

где $\{\cdot\}$ означает дробную долю указанного в скобках числа. Поскольку теперь значение X_n зависит от двух более ранних значений, период получающейся последовательности может оказаться больше m, но не больше $m^2 - 1$.

© А.В. Зорин, ННГУ, 2021

Обобщенный генератор Фибоначчи с запаздыванием III

Например, если начальными значениями X_n, $n < \max\{i, j\}$, выбраны нули, то получается последовательность из нулей и период оказывается равен единице.

Рассмотрим несколько подробней генератор вида

$$X_n = X_{n-i} + X_{n-j} \mod 2^e, \quad i < j. \tag{6}$$

и обсудим вопрос о длине периода.

Пусть $e = 1$. Тогда значениями X_n будут только 0 и 1, один двоичный разряд. Для поддержания работы генератора в этом случае надо знать ровно j последних значений последовательности. Это «внутреннее состояние» генератора занимает j бит и может принимать

© А.В. Зорин, ННГУ, 2021

Обобщенный генератор Фибоначчи с запаздыванием IV

2^j различных значений. Одно из них, соответствующее ровно j нулям, дает тривиальную нулевую последовательность. Оказывается, что при $j > 2$ все остальные $2^j - 1$ значений циклически переходят одно в другое. Таким образом, при $e = 1$ период равен $2^j - 1$.

Пусть теперь $e = 2$ и каждый X_n занимает два двоичных разряда. Легко увидеть, что младший разряд по-прежнему описывается соотношением (6) с $e = 1$ и, следовательно, имеет максимальный период $2^j - 1$. В то же время, эволюция старшего разряда есть суперпозиция двух процессов: *а*) сложение старших разрядов, порождающее последовательность вида (6) с $e = 1$ и максимальным

© А.В. Зорин, ННГУ, 2021

Обобщенный генератор Фибоначчи с запаздыванием V

периодом $2^j - 1$, и *б*) последовательность переносов, образующихся при сложении младших разрядов. Пусть $X'_0 = 1$, $X'_n = 0$, $n < j$. Такая последовательность моделирует появления 1 при переносе в результате сложения «в момент времени 0». Тогда и при $n \geq j$ единица будет снова появляться среди значений X'_n и образовывать последовательность с периодом $2^j - 1$. Поэтому при достаточно больших n значение старшего разряда X_n есть сумма старших разрядов X_{n-i} и X_{n-j} и набегающих волн от ранее выполненных переносов. Если за счет переносов добавляется четное число единиц, то по модулю 2 фактически прибавляется 0. Поэтому при $e = 2$ слагаемые за счет переносов будут образовывать последовательность периода $2 \times (2^j - 1)$. Другими

© А.В. Зорин, ННГУ, 2021

Обобщенный генератор Фибоначчи с запаздыванием VI

словами, происходит удвоение периода. Понятно, что и последовательность $\{X_n; n = 0, 1, \ldots\}$ будет иметь период $2(2^j - 1)$.

В общем случае максимальный период для рекуррентного соотношения (6) равен $2^{e-1}(2^j - 1)$. Известны несколько наборов чисел (i, j), $i < j$, для которых последовательность, порождаемая соотношением (6), имеет максимальный период:

$$(24, 55),\ (37, 100),\ (83, 258),\ (273, 607),\ (9739, 23209)$$

Условие на выбор начальных значений для генератора, при которых достигается максимальная длина периода, состоит в наличии нечетных чисел среди них.

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием I

Первый векторный обобщенный генератор Фибоначчи с запаздыванием

Пусть заданы векторы

$$X_0 = (X_0^{(1)}, X_0^{(2)}, \ldots, X_0^{(k)}), \quad X_1 = (X_1^{(1)}, X_1^{(2)}, \ldots, X_1^{(k)}), \quad \ldots$$
$$\ldots, \quad X_{k-1} = (X_{k-1}^{(1)}, X_{k-1}^{(2)}, \ldots, X_{k-1}^{(k)}).$$

Последовательно определяем

$$X_n = X_{n-i} \circ X_{n-j}, \quad n \geq \max\{i, j\}.$$

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием II

Рассмотрим генератор, задаваемый соотношением

$$X_n = X_{n-607} - X_{n-273} \mod 2^{31}, \quad n \geq 607.$$

Пусть допускаются векторные операции над векторами длины $k = 4$. Как было замечено выше, можно перейти к арифметике вещественных чисел:

$$U_n = \{U_{n-607} - U_{n-273}\}, \quad n \geq 607, \tag{7}$$

где $\{\cdot\}$ обозначает дробную долю числа, и $U_n \in (0, 1)$ для всех n. Тогда рекуррентное соотношение (7) может применяться параллельно и поэлементно к векторам

$$\mathcal{U}' = (U_{n-607}, U_{n-606}, U_{n-605}, U_{n-604}),$$
$$\mathcal{U}'' = (U_{n-273}, U_{n-272}, U_{n-271}, U_{n-270}).$$

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием III

с целью получения вектора $\mathcal{U} = (U_n, U_{n+1}, U_{n+2}, U_{n+3})$.
Заметим, что $152 \cdot 4 = 608 > 607$.
Мы будем поддерживать массив из $(152 + 1) \times 4 = 612$ чисел с плавающей точкой одинарной точности, который для наглядности удобно представить в виде таблицы:

$$\begin{array}{cccccc}
U_1 & U_5 & U_9 & \dots & U_{605} & U_{609} \\
U_2 & U_6 & U_{10} & \dots & U_{606} & U_{610} \\
U_3 & U_7 & U_{11} & \dots & U_{607} & U_{611} \\
U_4 & U_8 & U_{12} & \dots & U_{608} & U_{612}
\end{array}$$

Столбцы таблицы будут интерпретироваться как четырехкомпонентные векторы. Сам массив будет использоваться как кольцевой буфер.

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием IV

Второй векторный обобщенный генератор Фибоначчи с запаздыванием

Шаг 1. Заполнить массив чисел U_1, U_2, ..., U_{608}, произвольным образом. Например, первые 607 чисел заполняются с помощью линейного конгруэнтного генератора. Величины с U_{608} по U_{612} — по скалярной версии генератора Фибоначчи с запаздыванием.
Шаг 2. Скопировать последнюю колонку чисел на место первой колонки.

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием V

Шаг 3. Выбрать в качестве $\mathcal{U}'$ отрезок массива с U_5 по U_8, а в качестве $\mathcal{U}''$ — отрезок массива с U_{339} по U_{342}. Вычислить $\mathcal{U}$ и записать результат во второй столбец. Выбрать в качестве векторов $\mathcal{U}'$ и $\mathcal{U}''$ новые отрезки на 4 позиции правее. Если какой-то из отрезков, используемый для выбора значений $\mathcal{U}'$ или $\mathcal{U}''$ должен начаться в последнем, 153-м столбце, то участок для выбора сдвигается на 608 позиций влево, на первый столбец. Затем продолжается движение этого участка вправо. Так заполняются все столбцы со второго по 153-й. После этого возвращаемся к шагу 2.

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием VI

Listing 5: Генератор Фибоначчи с запаздыванием для соотношения $U_n = \{U_{n-607} - U_{n-273}\}$

```
typedef float v4sf
     __attribute__ ((vector_size (16)));
class lfrng{
  float seed[612];
  int loc;
  lfrng();
  void generate();
public:
  lfrng(int _seed);
  float next();
};
```

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием VII

```
inline v4sf submod(v4sf x, v4sf y){
  v4sf z;
  z = x-y;
  z += __builtin_ia32_andps(
   __builtin_ia32_cmpltps(z,(v4sf){0.,0.,0.,0.}),
      (v4sf){1.,1.,1.,1.});
  return z;
}

float lfrng::next(){
  float ans;
  ans = seed[loc];
  loc++;
  if(loc>607) generate();
```

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием VIII

```
  return ans;
};

lfrng::lfrng(int _seed){
  float intp;
  for(int i=0; i<607; i++){
    _seed = _seed*16535 & 0x7fffffff;
    seed[i]= (_seed+1.0)/((float)0x80000000);
  };
  for(int i=607; i<612; i++){
    float t=seed[i-607]-seed[i-273];
    if (t<0) seed[i]=1.0+t; else seed[i]=t;
  };
  loc=607;
```

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием IX

```
};

void lfrng::generate(){
  short i, j, k;
  *(v4sf*)seed = *(v4sf*)(seed+608);
  i=5;
  j=339;
  for(k=1; k<153; k++){
    *(v4sf*)(seed+4*k)= submod(
                __builtin_ia32_loadups(seed+i),
                __builtin_ia32_loadups(seed+j));
    i+=4; if(i>607) i-=608;
    j+=4; if(j>607) j-=608;
  }
```

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием X

```
  loc=0;
};
```

Listing 6: (Функция-обертка для подключения генератора Фибоначчи с запаздыванием к R)

```
// Rrngtest.cc - wrapper for use with R
#include <cstdlib>
#include <ctime>
#include <Rcpp.h>
#include "lfrng.h"

using namespace Rcpp;

lfrng rlf(time(0));
```

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием XI

```
RcppExport SEXP rlfrng(SEXP _sz)
{
  int sz = as<int>(_sz);
  Rcpp::NumericVector x(sz);
  for(int j=0; j<sz; j++)
    x[j] = rlf.next();
  return x;
}
```

Для компиляции (в Linux) надо экспортировать две переменные и воспользоваться интерфейсом R:

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием XII

```
export PKG_LIBS=`Rscript -e "Rcpp::LdFlags()"`
export PKG_CXXFLAGS="Rscript -e 'Rcpp::CxxFlags()'"
R CMD SHLIB -o Rrngtest.so Rrngtest.cc lfrng.cc
```

Теперь можно подключить динамическую библиотеку и «пробросить вызов» в нее:

Listing 7: (проверка равномерного распределения для одномерного закона по критерию Колмогорова–Смирнова и для трехмерного распределения по критерию χ^2 Пирсона)

```
library(Rcpp)
dyn.load("Rrngtest.so")
rlfrng <- function(n){
  .Call("rlfrng",as.integer(n));}
```

© А.В. Зорин, ННГУ, 2021

Векторный обобщенный генератор Фибоначчи с запаздыванием XIII

```
x <- rlfrng(10000)
hist(x, pr=TRUE, col="grey")
ks.test(x, punif)
x.3d <- data.frame( x1=x[seq(1, 10000, 3)],
                    x2=x[seq(2, 10000, 3)],
                    x3=x[seq(3, 10000, 3)] )
x.t3 <- table( lapply( x.3d, cut,
                       br=c(0, 0.25, 0.5, 0.75, 1)))
chisq.test( as.vector(x.t3) )
```

© А.В. Зорин, ННГУ, 2021

Методы перетасовки и комбинации I

Пусть псевдослучайные последовательности $U = \{U_0, U_1, \dots\}$ и $V = \{V_0, V_1, \dots\}$ со значениями в $[0, 1)$ порождаются *различными* алгоритмами.

Алгоритм перетасовывания для двух псевдослучайных последовательностей

Шаг 0. Выберем натуральное K и введем вспомогательные переменные A_0, A_1, ..., A_{K-1}.
Шаг 1. Положим $A_0 = U_0$, $A_1 = U_1$, ..., $A_{K-1} = U_{K-1}$.
Шаг 2. Пусть u и v — следующие числа в последовательностях U и V. Положим $j = [Kv]$ — целая часть от произведения Kv.
Шаг 3. Выдаем следующее число последовательности, A_j; присваиваем $A_j = u$. Возвращаемся к шагу 2.

© А.В. Зорин, ННГУ, 2021

Методы перетасовки и комбинации II

Алгоритм перетасовывания для одной последовательности

Выберем натуральное K и введем вспомогательные переменные A_0, A_1, ..., A_{K-1}.

Шаг 1. Положим $A_0 = U_0$, $A_1 = U_1$, ..., $A_{K-1} = U_{K-1}$.

Шаг 2. Выберем следующий член u последовательности U. Положим $j = [Ku]$.

Шаг 3. Возвращаем A_j; присваиваем $A_j = u$. Переходим к шагу 2.

© А.В. Зорин, ННГУ, 2021

Методы перетасовки и комбинации III

Другим популярным способом получения псевдослучайных последовательностей является комбинация нескольких последовательностей в одну. Рассмотрим две целочисленные псевдослучайные последовательности $\{X_0, X_1, \dots\}$ и $\{Y_0, Y_1, \dots\}$ со значениями в $\{0, 1, \dots, m-1\}$. Определим *комбинацию* этих последовательностей посредством соотношения

$$Z_n = X_n \circ Y_n \mod m, \quad n = 0, 1, \dots$$

Например, для псевдослучайных последовательностей $U = \{U_0, U_1, \dots\}$ и $V = \{V_0, V_1, \dots\}$ чисел из интервала $(0, 1)$ можно положить

$$W_n = \{U_n + V_n\}, \quad n = 0, 1, \dots.$$

© А.В. Зорин, ННГУ, 2021

Методы перетасовки и комбинации IV

Комбинация двух псевдослучайных последовательностей может быть реализована на векторном компьютере с использованием функции `submod`. Дополнительная вычислительная эффективность может быть достигнута путем включения кода, генерирующего последовательности U и V.

© А.В. Зорин, ННГУ, 2021

«Вихрь Мерсенна» I

Следующий генератор псевдослучайных чисел обладает некоторыми важными статистическими свойствами. Пусть w — число двоичных разрядов в машинном слове. Рассмотрим двоичное представление целого беззнакового числа $0 \leq X < 2^w$ как массив из w бит:

$$X = (X^{(w-1)} X^{(w-2)} \dots X^{(0)})_2.$$

Обозначим $X^{[k:l]}$ подвектор $(X^{(l)} \dots X^{(k)})$ вектора U. В основе генератора лежит рекуррентное соотношение

$$X_{k+n} = X_{k+m} \oplus (X_k^{(r:w-1)} | X_{k+1}^{(0:r-1)}) \otimes A, \quad k = 0, 1, \dots,$$

где $1 \leqslant m \leqslant n$ и $0 \leq r \leq w-1$ — целочисленные параметры, $\oplus$ обозначает сложение векторов по модулю 2, вертикальная черта означает конкатенацию двух векторов,

© А.В. Зорин, ННГУ, 2021

«Вихрь Мерсенна» II

A — матрица размера $w \times w$ с элементами 0 и 1, $\otimes$ — умножение матриц по модулю два. Начальные значения $X_0, X_1, \ldots, X_{n-1}$ выбираются произвольно.

Матрица A имеет вид

$$A = \begin{pmatrix} 0 & 1 & 0 & \ldots & 0 \\ 0 & 0 & 1 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ldots \\ 0 & 0 & 0 & \ldots & 1 \\ a_{w-1} & a_{w-2} & a_{w-3} & \ldots & a_0 \end{pmatrix},$$

так что вычисление произведения $X \otimes A$ может быть выполнено только с использованием побитовых операций

© А.В. Зорин, ННГУ, 2021

«Вихрь Мерсенна» III

сдвига вправо (shiftright) и сложения по модулю два: при $X^{(0)} = 0$ имеем

$$X \otimes A = \mathrm{shiftright}(X),$$

при $X^{(0)} = 1$ имеем

$$X \otimes A = \mathrm{shiftright}(X) \oplus (a_{w-1}a_{w-2}\ldots a_0)_2.$$

Операции вычисления подвектора и конкатенации векторов легко реализуются с помощью операций «логическое И» (**and**) и «логическое ИЛИ» (**or**). Далее операции $\oplus$ соответствует «логическое исключающее ИЛИ».
Таким образом, расчет по рекуррентной формуле производится только с использованием основных логических и побитовых операций.

© А.В. Зорин, ННГУ, 2021

«Вихрь Мерсенна» IV

Кроме того, оказывается полезным вместо X_{n+k} вычислять X'_{n+k} по формулам:

$$
\begin{aligned}
Y' &= X_{n+k} \oplus (X_{n+k} \gg u), \\
Y'' &= Y' \oplus ((Y' \ll s)\ B), \\
Y''' &= Y'' \oplus ((Y'' \ll t)\,\mathbf{and}\,C), \\
X'_{n+k} &= Y''' \oplus (Y''' \gg l),
\end{aligned}
$$

где l, s, t и u — целые числа, B, C — двоичные массивы длины w, операция $X \gg u$ означает сдвиг массива X вправо на u мест, операция $X \ll u$ — влево на u мест.

В теории генератора «Вихрь Мерсенна» доказывается, что максимальный период генератора равен $2^{nr} - 1$.

© А.В. Зорин, ННГУ, 2021

«Вихрь Мерсенна» V

Широко применяется вариант вихря Мерсенна MT19937 при значении параметров $w = 32$, $n = 624$, $m = 397$, $r = 31$, $u = 11$, $s = 7$, $t = 15$, $l = 18$,

$$(a_{31}a_{30}\ldots a_0) = (10011001000010001011000011011111)_2,$$
$$B = (10011101001011000101011010000000)_2,$$
$$C = (11101111110001100000000000000000)_2.$$

Период этого генератора весьма велик:
$2^{19968} - 1 \approx 9{,}267 \times 10^{6010}$.

© А.В. Зорин, ННГУ, 2021

Задания для самостоятельной работы

1. Сгенерировать последовательность псевдослучайных чисел, порожденную рекуррентными соотношениями

$$Z_n = X_n \oplus Y_n, \qquad U_n = Z_n/M,$$
$$X_n = X_{n-k} + X_{n-k} \mod M,$$
$$Y_n = Y_{n-k} + Y_{n-k} \mod M,$$

проверить одномерные и совместные распределения на равномерность.

© А.В. Зорин, ННГУ, 2021